

Detailed Development Flow (What → How → Where)

Phase 1 — Foundation (Do This First)

Step	What to Create	How	Folder / File
1.1 	Role enum	Done	enums/Role.java
1.2	User entity	JPA entity with id, name, email, password, role	entity/User.java
1.3	UserRepository	extends JpaRepository<User, Long>	repository/UserRepository.java
1.4	Signup DTO	SignupRequest (email, password, role)	dto/request/SignupRequest.java
1.5	Auth service	Register + hash password (BCrypt)	service/auth/AuthService.java
1.6	Auth controller	POST /api/auth/signup, POST /api/auth/login	controller/auth/AuthController.java
1.7	Security config	JWT filter, role-based access	security/ + config/SecurityConfig.java
1.8	React signup page	Form → call /api/auth/signup	frontend/src/pages/auth/Signup.jsx

Step 1 goal: Candidate/Recruiter signup with profile creation and resume upload endpoint stub.

Phase 2 — Resume Module (Flow Step 2)

Step	What	How	Where
2.1	Resume entity	userId, filePath, parsedJson, uploadedAt	entity/Resume.java
2.2	Upload API	POST /api/candidate/resume/upload (MultipartFile)	controller/candidate/ResumeController.java
2.3	File storage	Save to uploads/resumes/	service/resume/ResumeStorageService.java
2.4	Parsing service	Extract skills, experience, education (Regex/NLP/Spring AI)	service/resume/ResumeParsingService.java
2.5	Parsed profile DTO	Return structured JSON to frontend	dto/response/ParsedResumeResponse.java

Phase 3 — JD Matching (Flow Step 3)

Step	What	How	Where
3.1	Job entity	title, description, skills, recruiterId	entity/Job.java
3.2	Recruiter post job API	POST /api/recruiter/jobs	controller/recruiter/JobController.java
3.3	Match service	Compare resume skills vs JD (Spring AI similarity)	service/job/JdMatchingService.java
3.4	Match score API	GET /api/candidate/jobs/match → % score	controller/candidate/JobMatchController.java

Phase 4 — Aptitude Tests (Flow Steps 4–5)

Step	What	How	Where
4.1	Enums	TestSection (QUANT, LOGICAL, VERBAL, CODING)	enums/TestSection.java
4.2	Question, TestAttempt, Answer entities	Question bank + attempt tracking	entity/
4.3	Question bank seed	SQL in db/migration/V2__seed_questions.sql	resources/db/migration/
4.4	Start test API	Create attempt, return questions + timer	service/aptitude/TestEngineService.java
4.5	Timed engine	Server-side expiry + auto-submit	service/aptitude/TimerService.java
4.6	Auto evaluation	Grade MCQ/coding answers section-wise	service/aptitude/EvaluationService.java
4.7	React test UI	Section tabs + countdown timer	pages/candidate/AptitudeTest.jsx

Phase 5 — Dashboards (Flow Step 6)

Candidate dashboard

Feature	API	Frontend
Job applications	GET /api/candidate/applications	pages/candidate/Dashboard.jsx
Match scores	GET /api/candidate/match-scores	Chart/cards

Feature	API	Frontend
Test results	GET /api/candidate/test-results	Section-wise scores
Recruiter dashboard		
Feature	API	Frontend
Post jobs	POST /api/recruiter/jobs	pages/recruiter/PostJob.jsx
View applicants	GET /api/recruiter/jobs/{id}/applicants	pages/recruiter/Applicants.jsx
Shortlist	PATCH /api/recruiter/applicants/{id}/shortlist	Shortlist button
Resume screening	Auto-filter by match score	service/resume/ResumeScreeningService.java
Reports	GET /api/recruiter/reports	pages/recruiter/Reports.jsx

Phase 6 — Interview Prep (Flow Steps 7–8)

Step	What	How	Where
6.1	InterviewQuestion entity	role, skill, difficulty, questionText	entity/InterviewQuestion.java
6.2	Question bank API	CRUD for interview questions	controller/candidate/InterviewController.java

Step	What	How	Where
6.3	AI question generator	Spring AI → generate from JD + resume	service/ai/InterviewQuestionGenerator.java
6.4	AI mock interview	Chat-style Q&A with evaluation	service/ai/MockInterviewService.java

Phase 7 — Resume Template Builder

Step	What	How	Where
7.1	HTML templates	ATS-friendly layouts	resources/templates/resume/
7.2	Builder API	Form data → PDF (iText/OpenPDF)	service/resume/ResumeTemplateService.java
7.3	React builder form	Fill sections → preview → download	pages/candidate/ResumeBuilder.jsx

Recommended Build Order (Sprint Plan)

1. Auth & Signup
2. Resume Upload
3. Resume Parsing
4. Job Posting
5. JD Matching
6. Aptitude Tests
7. Auto Evaluation
8. Dashboards
9. Recruiter Flow
10. Interview AI

Sprint	Focus	Deliverable
Sprint 1	Auth + DB setup	Signup/login works for both roles
Sprint 2	Resume upload + parsing	Upload PDF → get skills JSON

Sprint	Focus	Deliverable
Sprint 3	Jobs + matching	Recruiter posts job, candidate sees match %
Sprint 4	Aptitude engine	Timed section-wise test + auto score
Sprint 5	Dashboards	Candidate + recruiter views
Sprint 6	AI features	Interview questions + mock interview
Sprint 7	Resume builder + reports	PDF export + recruitment reports

Next Immediate Step (Step 1.2)

Create the User entity:

```
// entity/User.java
```

```
@Entity
```

```
@Table(name = "users")
```

```
public class User {
```

```
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String name;
```

```
    @Column(unique = true) private String email;
```

```
    private String password;
```

```
    @Enumerated(EnumType.STRING) private Role role;
```

```
    // getters/setters
```

```
}
```

Also add to pom.xml:

- spring-boot-starter-data-jpa
- spring-boot-starter-validation

And configure MySQL in application.properties.